

第六章

(第 3、4、5、8、12、13、19 题)

3. 某计算机主存最大寻址空间为 4GB, 按字节编址, 假定用 $64\text{M} \times 8$ 位的具有 8 个位平面的 DRAM 芯片组成容量为 512MB、传输宽度为 64 位的内存条。回答下列问题。

(1) 每个内存条需要多少个 DRAM 芯片?

(2) 构建容量为 2GB 的主存时, 需要几个内存条?

(3) 主存地址共有多少位? 其中哪几位用作 DRAM 芯片内地址? 哪几位为 DRAM 芯片内的行地址? 哪几位为 DRAM 芯片内的列地址? 哪几位用于选择芯片?

参考答案:

(1) $512\text{MB} / (64\text{M} \times 8 \text{ 位}) = 8 \text{ 片 DRAM}$ 。

(2) $2\text{GB} / 512\text{MB} = 4 \text{ 个内存条}$ 。

(3) 因为是按字节编址, 所以主存地址共 32 位: $A_{31}A_{30} \cdots A_{25} \cdots A_1A_0$ 。因为每次在总线上传输 64 位数据, 因此, 内存条内 8 个芯片同时进行读写, 每个芯片有 8 个位平面, 因而同时读出 64 位数据。芯片按交叉编址方式组织, 每个内存条有 512MB 容量, 故内存条内内存存储单元的地址有 29 位, 即 $A_{28} \cdots A_4A_3A_2A_1A_0$ 。其中, $A_2A_1A_0$ 表示芯片号, $A_{28} \cdots A_4A_3$ 表示芯片内存存储单元的地址。其中芯片内的行地址为 $A_{28} \cdots A_{16}$, 列地址为 $A_{15} \cdots A_3$ 。

4. 某计算机按字节编址, 其中已配有 0000H~7FFFH 的 ROM 区域, 现在再用 $16\text{K} \times 4$ 位的 RAM 芯片形成 $32\text{K} \times 8$ 位的存储区域, CPU 地址线为 $A_0 \sim A_{15}$ 。回答下列问题。

(1) RAM 区地址范围是什么? 共需多少 RAM 芯片? 地址线中哪一位用来区分 ROM 区和 RAM 区?

(2) 假定 CPU 地址线改为 24 根, 地址范围 000000H~007FFFH 为 ROM 区, 剩下的所有地址空间都用 $16\text{K} \times 4$ 位的 RAM 芯片配置, 则需要多少个这样的 RAM 芯片?

参考答案:

(1) 地址共 16 位, 故存储器地址空间为 0000H~FFFFH, 其中, 8000H~FFFFH 为 RAM 区, 共 $2^{15} = 32\text{K}$ 个单元, 其空间大小为 32KB, 故需要 $16\text{K} \times 4$ 位的芯片数为 $32\text{KB} / (16\text{K} \times 4 \text{ 位}) = 2 \times 2 = 4 \text{ 片}$ 。

因为 ROM 区在 0000H~7FFFH, RAM 区在 8000H~FFFFH, 所以可通过最高位地址 A_{15} 来区分, 当 A_{15} 为 0 时选中 ROM 芯片; 为 1 时选中 RAM 芯片, 此时, 根据 A_{14} 进行译码, 得到两个译码信号, 分别用于两组字扩展芯片的片选信号。

(2) 若 CPU 地址线为 24 根, ROM 区为 000000H~007FFFH, 则 ROM 区大小为 32KB, 而总大小为 $16\text{MB} = 2^{14}\text{KB} = 512 \times 32\text{KB}$, 所以 RAM 区大小为 $511 \times 32\text{KB}$, 共需使用 $16\text{K} \times 4$ 位的 RAM 芯片数为 $511 \times 32\text{KB} / (16\text{K} \times 4 \text{ 位}) = 511 \times 2 \times 2 \text{ 个芯片}$ 。

5. 假设一个程序重复完成将磁盘上一个 4KB 的数据块读出, 进行相应处理后, 写回到磁盘的另外一个数据区。各数据块内信息在磁盘上连续存放, 数据块随机位于磁盘的一个磁道上。磁盘转速为 7200 r/min, 平均寻道时间为 10ms, 磁盘最大内部数据传输率为 40MB/s, 磁盘控制器的开销为 2ms, 没有其他程序使用磁盘和处理器, 并且磁盘读写操作和磁盘数据的处理时间不重叠。若程序对磁盘数据的处理需要 20 000 个时钟周期, 处理器时钟频率为 500MHz, 则该程序完成一次数据块“读出一处理一写回”操作所需的时间为多少? 每秒钟可以完成多少次这样的数据块操作?

参考答案:

磁盘转一圈的时间为 $10^3 / (7200/60) = 8.33\text{ms}$, 故平均旋转等待时间为 $8.33/2 = 4.17\text{ms}$ 。

数据传输时间: $10^3 \times 4\text{KB} / 40\text{MB} = 0.1024\text{ms}$

平均存取时间 T : 控制器开销 + 寻道时间 + 旋转等待时间 + 数据传输时间
 $= 2\text{ms} + 10\text{ms} + 4.17\text{ms} + 0.1024\text{ms} = 16.27\text{ms}$

数据块的处理时间: $20000/500\text{M} = 0.04\text{ms}$

完成一次数据块的“读出-处理-写回”操作时间: $16.27\text{ms} \times 2 + 0.04\text{ms} = 32.58\text{ms}$

每秒中可以完成这样的数据块操作次数大约为: $1000\text{ms}/32.58\text{ms} = 30$ 次

8. 假设某计算机主存地址空间大小为1GB, 按字节编址, cache的数据区(即不包括标记、有效位等存储区)有64KB, 块大小为128B, 采用直接映射和全写(write-through)方式。回答下列问题。

(1) 主存地址多少位? 如何划分? 要求说明每个字段的含义、位数和在主存地址中的位置。

(2) cache的总容量为多少位?

参考答案:

(1) 主存空间大小为 1GB, 按字节编址, 说明主存地址为 30 位。cache 共有 $64\text{KB}/128\text{B} = 512$ 行, 因此, 行索引(行号)为 9 位; 块大小 128 字节, 说明块内地址为 7 位。因此, 30 位主存地址中, 高 14 位为标记(Tag); 中间 9 位为行索引; 低 7 位为块内地址。

(2) 因为采用直接映射, 所以cache中无需替换算法所需控制位, 全写方式下也无需修改(dirty)位, 而标志位和有效位总是必须有的, 所以, cache总容量为 $512 \times (128 \times 8 + 14 + 1) = 519.5\text{K}$ 位。

12. 以下是计算两个向量点积的程序段:

```
float dotproduct (float x[8], float y[8])
{
    float sum = 0.0;
    int i;
    for (i = 0; i < 8; i++)    sum += x[i] * y[i];
    return sum;
}
```

回答下列问题或完成下列任务。

(1) 试分析该段代码中访问数组 x 和 y 的时间局部性和空间局部性, 并推断命中率的高低。

(2) 假设该段程序运行的计算机中的数据 cache 采用直接映射方式, 其数据区容量为 32B, 主存块大小为 16B; 编译程序将变量 sum 和 i 分配给寄存器, 数组 x 存放在 40H 开始的主存区域, 数组 y 紧跟在 x 后。试计算该程序中数据访问的命中率, 要求说明每次访问时 cache 的命中情况。

(3) 将 (2) 中的数据 cache 改用 2 路组相联映射方式, 主存块大小改为 8B, 其他条件不变, 则该程序数据访问的命中率是多少?

(4) 在 (2) 中条件不变的情况下, 将数组 x 定义为 float x[12], 则数据访问的命中率又是多少?

参考答案:

(1) 数组 x 和 y 都按存放顺序访问, 不考虑映射的情况下, 空间局部性都较好, 但都只被访问一次, 故没有时间局部性。命中率的高低与块大小、映射方式等都有关系, 所以, 无法推断命中率的高低。

(2) cache 采用直接映射方式, 块大小为 16 字节, 数据区大小为 32 字节, 故 cache 共有 2 行。数组 x 的 8 个元素(共 32B)分别存放在主存 40H 开始的 32 个单元中, 共有 2 个主存块, 其中 $x[0] \sim x[3]$ 在第 4 块, $x[4] \sim x[7]$ 在第 5 块中; 数组 y 的 8 个元素(共 32B)分别在主存第 6 块和第 7 块中。所以, $x[0] \sim x[3]$ 和 $y[0] \sim y[3]$ 都映射到 cache 第 0 行;

$x[4] \sim x[7]$ 和 $y[4] \sim y[7]$ 都映射到 cache 第 1 行。

Cache	第 0-3 次 循环	第 4-7 次 循环
-------	---------------	---------------

第 0 行	x[0-3] , y[0-3]	
第 1 行		x[4-7] , y[4-7]

每调入一块，装入 4 个数组元素，因为 x[i]和 y[i]总是映射到同一行，相互淘汰对方，故每次都都不命中，命中率为 0。

(3) 改用 2 路组相联，块大小为 8B，则 cache 共有 4 行，每组两行，共两组。

数组 x 有 4 个主存块，x[0]~x[1]、x[2]~x[3]，x[4]~x[5]，x[6]~x[7]分别在第 8~11 块中；

数组 y 有 4 个主存块，y[0]~y[1]、y[2]~y[3]，y[4]~y[5]，y[6]~y[7]分别在第 12~15 块中；

cache	第 0 行	第 1 行
第 0 组	x[0-1], x[4-5]	y[0-1], y[4-5]
第 1 组	x[2-3], x[6-7]	y[2-3], y[6-7]

每调入一块，装入两个数组元素，第二个数组元素的访问总是命中，故命中率为 50%。

(4) 若 (2) 中条件不变，数组 x 定义了 12 个元素，共有 48B，使得 y 从第 7 块开始，因而，x[i]和 y[i]就不会映射到同一个 cache 行中，即：x[0]~x[3]在第 4 块，x[4]~x[7]在第 5 块，x[8]~x[11]在第 6 块中，y[0]~y[3]在第 7 块，y[4]~x[7]在第 8 块。

cache	第 0-3 次 循环	第 4-7 次 循环
第 0 行	x[0-3]	y[4-7]
第 1 行	y[0-3]	x[4-7]

每调入一块，装入 4 个数组元素，第一个元素不命中，后面 3 个总命中，故命中率为 75%。

13. 以下是对矩阵进行转置的程序段：

```
typedef int array[4][4];
void transpose(array dst, array src)
{
    int i, j;
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4; j++)    dst[j][i] = src[i][j];
}
```

假设该段程序运行的计算机中 sizeof(int)=4，且只有一级 cache，其中 L1 data cache 的数据区大小为 32B，采用直接映射、回写方式，块大小为 16B，初始为空。数组 dst 从地址 0x804c000 开始存放，数组 src 从地址 0x804c010H 开始存放。仿照例子填写下表，说明数组元素 src[row][col]和 dst[row][col]各自映射到 cache 哪一行，访问是命中 (hit) 还是缺失 (miss)。若 L1 data cache 的数据区容量改为 128B 时，重新填写表中内容。

	src 数组				dst 数组			
	col=0	col=1	col=2	col=3	col=0	col=1	col=2	col=3
row=0	0/miss							
row=1								

ro w=2								
ro w=3								

参考答案：

从程序来看，数组访问过程如下：

src[0][0]、dst[0][0]、src[0][1]、dst[1][0]、src[0][2]、dst[2][0]、src[0][3]、dst[3][0]

src[1][0]、dst[0][1]、src[1][1]、dst[1][1]、src[1][2]、dst[2][1]、src[1][3]、dst[3][1]

src[2][0]、dst[0][2]、src[2][1]、dst[1][2]、src[2][2]、dst[2][2]、src[2][3]、dst[3][2]

src[3][0]、dst[0][3]、src[3][1]、dst[1][3]、src[3][2]、dst[2][3]、src[3][3]、dst[3][3]

因为块大小为 16B，每个数组元素有 4 个字节，所以 4 个数组元素占一个主存块，因此每次总是调入 4 个数组元素到 cache 的一行。

当数据区容量为 32B 时，L1 data cache 中共有 2 行。数组元素 dst[0][i]、dst[2][i]、src[0][i]、src[2][i] (i=0~3)都映射到 cache 第 0 行，数组元素 dst[1][i]、dst[3][i]、src[1][i]、src[3][i] (i=0~3)都映射到 cache 第 1 行。因此，从上述访问过程来看，src[0][0]所在的一个主存块（即存放 src[0][i] (i=0~3)四个数组元素）刚调入 cache 后，dst[0][0]所在主存块又把 src[0][0]替换掉了。……

当数据区容量为128B时，L1 data cache中共有8行。数组元素dst[0][i]、dst[1][i]、dst[2][i]、dst[3][i]、src[0][i]、src[1][i]、src[2][i]、src[3][i] (i=0~3) 分别映射到cache第0、1、2、3、4、5、6、7行。因此，不会发生数组元素的替换。每次总是第一个数组元素不命中，后面三个数组元素都命中。

	src 数组				dst 数组			
32 B	col=0	col=1	col=2	col=3	col=0	col=1	col=2	col=3
ro w=0	0/miss	0/miss	0/hit	0/miss	0/miss	0/miss	0/miss	0/miss
ro w=1	1/miss	1/hit	1/miss	1/hit	1/miss	1/miss	1/miss	1/miss
ro w=2	0/miss	0/miss	0/hit	0/miss	0/miss	0/miss	0/miss	0/miss
ro w=3	1/miss	1/hit	1/miss	1/hit	1/miss	1/miss	1/miss	1/miss
	src 数组				dst 数组			
128B	col=0	col=1	col=2	col=3	col=0	col=1	col=2	col=3
ro w=0	4/miss	4/hit	4/hit	4/hit	0/miss	0/hit	0/hit	0/hit
ro w=1	5/miss	5/hit	5/hit	5/hit	1/miss	1/hit	1/hit	1/hit
ro w=2	6/miss	6/hit	6/hit	6/hit	2/miss	2/hit	2/hit	2/hit
ro w=3	7/miss	7/hit	7/hit	7/hit	3/miss	3/hit	3/hit	3/hit

19. 假定某处理器带有一个数据区容量为256B的cache，其主存块大小为32B。以下C语言程序段运行在该处理器上，设sizeof(int)=4，编译器将变量i, j, c, s都分配在通用寄存器中，因此，只要考虑数组元素的访问情况。为简化问题，假定数组a从一个主存块开始处存放。若cache采用直接映射方式，则当s=64和s=63时，缺失率分别为多少？若cache采用2路组相联映射方式，则当s=64和s=63时，缺失率又分别为多少？

```
int i, j, c, s, a[128];
.....
for ( i = 0; i < 10000; i++ )
    for ( j = 0; j < 128; j=j+s )
        c = a[j];
```

参考答案：

已知块大小为 32B，占 8 个数组元素，cache 容量为 256B = 8 行 × 32B/行，仅考虑数组访问情况。

a[0]~a[7]在同一块；a[8]~a[15]在同一块；.....

a[0]~a[63]为前 8 块（同一个块群），.....

- 1) 直接映射，s=64: 访存顺序为 a[0]、a[64]、a[0]、a[64]、... ..，共循环 10000 次。这两个元素所在的块总是被映射到同一个 cache 行中，每次都会发生冲突，因此缺失率为 100%。

因为 a[0]所在块号（0#）与 a[64]所在块号（8#）总是相差 8，即模 8 同余。

- 2) 直接映射，s=63: 访存顺序为 a[0]、a[63]、a[126]、a[0]、a[63]、a[126]、... ..共循环 10000 次。这三个元素中后面两个元素因为映射到同一个 cache 行中，因此每次都会发生冲突，而 a[0]不会发生冲突，故缺失率为 67%。

因为 a[63]所在块号（= [63/8]_{取整}=7#）与 a[126]所在块号（=[126/8]_{取整}=15#）模 8 同余。

- 3) 2-路组相联，s=64: 访存顺序为 a[0]、a[64]、a[0]、a[64]、... ..，共循环 10000 次。这两个元素虽然映射到同一个 cache 组，但可以放在该组不同 cache 行中，所以不会发生冲突，缺失率几乎为 0（仅最开始的两次缺失）。

a[0]所在块号（0#）与 a[64]所在块号（8#）模 4 同余。

- 4) 2-路组相联，s=63: 访存顺序为 a[0]、a[63]、a[126]、a[0]、a[63]、a[126]、... ..共循环 10000 次。这三个元素中后面两个元素虽映射到同一个 cache 组中，但可放在不同 cache 行中，而 a[0]不会发生冲突，故缺失率大约为 0（仅最开始的三次缺失）。

a[63]所在块号（= [63/8]_{取整}=7#）与 a[126]所在块号（=[126/8]_{取整}=15#）模 4 同余。

（假定 s=32 和 s=31）

已知块大小为 32B，cache 容量为 256B = 8 行 × 32B/行，仅考虑数组访问情况。

- 1) 直接映射，s=32: 访存顺序为 a[0]、a[32]、a[64]、a[96]、a[0]、a[32]、.....，共循环 10000 次。这四个元素中 a[0]和 a[64]、a[32]和 a[96]被映射到同一个 cache 行中，每次都会发生冲突，因此缺失率为 100%。
 $[0/8] \bmod 8 = [64/8] \bmod 8 = 0$, $[32/8] \bmod 8 = [96/8] \bmod 8 = 4$

- 2) 直接映射，s=31: 访存顺序为 a[0]、a[31]、a[62]、a[93]、a[124]、a[0]、a[31]、a[62]、.....共循环 10000 次。这五个元素中，a[31]和 a[93]、a[62]和 a[124]被映射到同一个 cache 行中，每次都会发生冲突，而 a[0]不会发生冲突，故缺失率为 4/5=80%。

$[0/8] \bmod 8 = 0$, $[31/8] \bmod 8 = [93/8] \bmod 8 = 3$, $[62/8] \bmod 8 = [124/8] \bmod 8 = 7$

- 3) 2-路组相联，s=32: 访存顺序为 a[0]、a[32]、a[64]、a[96]、a[0]、a[32]、.....，共循环 10000 次。这四个元素都映射到同一个 cache 组中，因为每组只有两行，所以四个元素每次都会发生冲突，缺失率大约为 100%。

$[0/8] \bmod 4 = [32/8] \bmod 4 = [64/8] \bmod 4 = [96/8] \bmod 4 = 0$

- 4) 2-路组相联，s=31: 访存顺序为 a[0]、a[31]、a[62]、a[93]、a[124]、a[0]、a[31]、a[62]、.....共循环 10000 次。这五个元素中，后面四个元素都映射到同一个 cache 组中，虽可放在不同 cache 行中，但只有两

行，故每次都发生冲突，而 $a[0]$ 不会发生冲突，故缺失率为 $4/5=80\%$ 。

$$[0/8] \bmod 4=0, [31/8] \bmod 4 = [62/8] \bmod 4 = [93/8] \bmod 4 = [124/8] \bmod 4 = 3$$